Karlsruhe University of Applied Sciences

Department of Computer Science and Business Information Systems

Business Information Systems

BACHELOR´S THESIS

Efficiency vs. Performance in Green AI: An Empirical Investigation of Energy Consumption, Carbon Footprint, and Predictive Accuracy of Classification Algorithms at Varying Dataset Sizes.

Author	Eron Qorri
Matriculation number	85383
Workplace	Hochschule Karlsruhe
First Advisor	Prof. Dr. rer. nat. Christine Preisach
Second Advisor	Prof. Dr. Reimar Hofmann
Closing Date	31 July, 2026

31 July, 2026

Chairman of the Examination Board

Contents

Chapter 1

Introduction

1.1 Problem statement and societal relevance

The current trend in machine learning is moving towards increasingly large models and datasets [1]. Although deep learning approaches often achieve the highest predictive accuracy, their energy demand and associated CO_2 emissions increase disproportionately [2, 3]. In an era marked by strict sustainability guidelines and rising energy costs, a critical question arises: At what point is a computationally "simple" model ecologically more viable than a highly complex one?

The primary objective of this thesis is to determine whether and when it is worthwhile to deploy more complex models, considering not only their predictive accuracy but also their associated energy consumption. To achieve this, a Logistic Regression, a Random Forest, an XGBoost model and a Multilayer Perceptron are compared under equal conditions across three datasets of varying sizes. This comparative analysis aims to provide a clearer understanding of the thresholds at which simpler models represent the better choice. Additionally, the study investigates how modifying a Multilayer Perceptron architecture, specifically varying the number of neurons and hidden layers, directly impacts energy consumption and CO_2 emissions.

To address the problem statement and achieve the outlined objectives, this thesis investigates the overarching question: *How does the relationship between energy consumption and predictive accuracy change for Logistic Regression, Random Forest, XGBoost, and Multilayer Perceptron when model complexity and dataset size are systematically varied?*

Specifically, the following sub-questions are examined:

- How do energy consumption (in kWh) and CO_2 emissions change when scaling a deep learning architecture (variation of neuron count)?
- Under which conditions (dataset size, model complexity) does the resource consumption of complex models exceed the achievable accuracy gain compared to simpler algorithms?

- How does the energy consumption differ between CPU-based training (Random Forest) and GPU-accelerated training (XGBoost, MLP) for the same classification task?

1.2 Motivation: When does a simpler model pay off?

1.3 Research questions and objectives

1.4 Structure of the Thesis

The remainder of this thesis is structured as follows: Chapter 2 provides the theoretical background, distinguishing between Green AI and Red AI, and introducing the relevant sustainability and classification metrics. Chapter 3 details the methodology, including dataset selection and the experimental hardware setup. Chapter 4 outlines the implementation process, focusing on data preprocessing, model training, and the logging of resource consumption. The empirical results are analyzed in Chapter 5, evaluating both performance and resource utilization across scaling dataset sizes. Chapter 6 discusses the findings, highlighting the economic and environmental implications. Finally, Chapter 7 concludes the thesis and provides actionable recommendations for sustainable machine learning practices.

$$S = \frac{F_1}{1 + \lambda_1 \cdot t_{\text{scaled}} + \lambda_2 \cdot c_{\text{scaled}}}$$

Chapter 2

Theoretical Background

2.1 Metrics

2.2 CodeCarbon

CodeCarbon is an open-source Python library for measuring the carbon footprint of code running on local hardware. Emissions are calculated as the product of two factors: the energy consumed by the hardware in kWh, and the carbon intensity of the local electricity grid in gCO_2eq/kWh. Carbon intensity is determined based on the country in which the experiment is run, using data from Our World in Data. The resulting emissions are expressed in kilograms of CO_2-equivalents ($kgCO_2eq$), a standardized measure that accounts for the global warming potential of different greenhouse gases [3].

Power consumption is sampled at 15-second intervals. Rather than relying on instantaneous power readings, *CodeCarbon* derives average power from accumulated hardware energy counters. Instead, it computes the energy delta between two consecutive measurements and divides by the elapsed time to derive average power. This approach is more robust against short-term fluctuations and provides a stable average over each measurement interval [4]. For NVIDIA GPUs, energy is read directly from hardware counters via the `nvidia-ml-py` library, providing accurate device-level measurements. RAM is estimated by *CodeCarbon*, specifically using a flat value of 5 watts per DIMM slot.

CPU measurement depends on the operating system and processor. On Linux, CodeCarbon reads energy directly from *RAPL* (Running Average Power Limit) hardware counters, which support both Intel and AMD processors [5]. On Windows, however, *RAPL* is not accessible. *CodeCarbon* therefore falls back to a TDP-based estimation: it identifies the CPU model from a database of over 2000 processors and assumes an average consumption of 50% of TDP. On systems where direct CPU energy measurement is unavailable, this TDP-based fallback may introduce inaccuracies, as TDP represents peak power rather than typical operating consumption.

Chapter 3

Related Work

Chapter 4

Experimental Design

This chapter outlines the experimental design used to evaluate the ecological efficiency of different machine learning classification algorithms. It describes the datasets, models, evaluation metrics, hyperparameter tuning and measurement setup that form the basis of this empirical analysis.

4.1 Research Questions

The main research question that this study aims to answer is: *How does the trade-off between energy consumption and predictive performance vary across Random Forest, XGBoost and MLP models when the complexity of the models and the size of the dataset are systematically varied?* The following sub-questions will also be addressed in the course of this thesis:

- Under what conditions do the resource consumption of more complex models exceed the achievable accuracy gain over simpler baselines?
- How does energy consumption and CO2 output scale with increasing neural network complexity (number of layers and neurons)?
- How does energy consumption differ between CPU-based and GPU-based training for equivalent tasks?
- How does reducing the number of training instances on large datasets affect model accuracy and energy consumption across different algorithms?

To answer these questions multiple classification algorithms and datasets were used in the experimental phase of this empirical analysis.

4.2 Datasets

Dataset size is a primary driver of both computational demand and energy consumption during model training [6]. To evaluate how this relationship varies across different scales,

three classification datasets were selected that span several orders of magnitude in size. Classification was chosen as the task type since it is well-established across all selected models and allows for direct comparison of predictive performance via standardized metrics such as accuracy and F1-score. The datasets are summarized in Table 4.1.

Table 4.1: Overview of datasets used in this analysis

Dataset	**Instances**	**Features**	**Classes**	**Scale**
Wine [7]	178	13	3	Small
Default of Credit Card Clients [8]	30,000	23	2	Medium
HIGGS [9]	11,000,000	28	2	Large

The *Wine* dataset contains 178 instances across 13 features, where each feature describes a chemical property of wines from the same region in Italy, derived from three different cultivars.

The *Default of Credit Card Clients* (hereafter: *Credit*) dataset contains 30,000 instances across 23 features and focuses on predicting payment default among credit card clients in Taiwan.

The *HIGGS* dataset is the largest dataset used in this study, comprising 11,000,000 instances across 28 features, seven of which are high-level features derived by physicists to help discriminate between the two classes [10]. The classification goal is to distinguish between signal events produced by Higgs bosons and background noise processes.

These three datasets vary significantly in size, enabling a systematic evaluation of how model performance and energy consumption scale across different levels of data complexity.

4.3 Models

Selecting models of varying complexity is central to answering the research questions of this study. A model that achieves competitive accuracy with significantly lower energy consumption represents a more ecologically efficient choice, but this trade-off can only be quantified if the comparison spans a meaningful range of complexity. For this reason, five classification models were selected, ranging from a linear baseline to tree-based ensemble methods and a neural network, covering low, medium, and high computational demand respectively.

Table 4.2: Overview of classification models used in this study

Model	**Type**	**Device**	**Role**
Logistic Regression [11]	Linear	CPU	Baseline
Random Forest [12]	Ensemble	CPU	Mid-complexity
XGBoost [13]	Gradient Boosting	CPU & GPU	Mid-complexity
Multilayer Perceptron	Neural Network	GPU	High-complexity

Logistic Regression serves as the lightweight baseline in this study. Despite its simplicity as a linear classifier, it often achieves competitive accuracy, making it well suited to quantifying whether the additional resource consumption of more complex models yields a meaningful gain in predictive performance.

Random Forest was selected as a mid-complexity representative of tree-based ensemble methods, as extensively described in Section 2.x . Relevant to this study, Random Forest does not support GPU-accelerated training and is therefore run exclusively on the CPU.

Building on the tree-based approach, XGBoost was chosen as a second mid-complexity model, with the key distinction that it also supports GPU-accelerated training. This makes it possible to directly compare the energy consumption and training time of CPU- and GPU-based tree ensemble methods under identical conditions, which is central to one of the research questions of this study.

The MLP was selected as the most complex model, representing the deep learning category. Its highly configurable architecture, particularly the number of hidden layers and neurons per layer, makes it well suited for the architectural complexity experiments conducted in this study, where these parameters are systematically varied to assess their impact on predictive performance and energy consumption. A more detailed description of the MLP architecture is provided in Section 2.x.

Together, the selected models cover a broad spectrum of algorithmic complexity, from a linear baseline to gradient boosting and deep learning, enabling a meaningful comparison of both predictive performance and energy efficiency across fundamentally different model types. The following section defines the metrics used to evaluate these dimensions.

4.4 Evaluation Metrics

To answer the research questions of this study, two categories of metrics were employed: classification performance and resource consumption. Accuracy and weighted F1-Score were selected to measure predictive performance, as they provide a standardized basis for comparing models across datasets of varying class distributions, as further described in Section 2.1. Energy consumption in kWh and CO_2 emissions in kg were chosen as the primary resource metrics, directly reflecting the ecological cost of model training. It should be noted that all experiments were conducted on a single hardware setup, which is described in detail in Section 5.1.1.

Training time in seconds was included as a secondary resource metric, as it captures computational demand independently of hardware-specific power draw. Furthermore, inference time was measured across all experimental setups to account for the operational footprint. In large-scale deployment scenarios, the cumulative energy demand of repeated predictions can often rival or even exceed the initial training costs [14]. Together, these metrics enable a direct assessment of whether gains in predictive performance justify the associated increase in resource consumption.

Inspired by the efficiency framing proposed in Schwartz et al., a custom composite metric was developed for this study, named the *Efficiency-Weighted F1* (EWF1), defined as:

$$\text{EWF1} = \frac{F_1}{1 + \lambda_1 \cdot \hat{t} + \lambda_2 \cdot \hat{c}} \tag{4.1}$$

where $\hat{t}$ and $\hat{c}$ denote the min-max normalized training time and CO2 emissions per dataset respectively. The weights were set to $\lambda_1 = 0.5$ and $\lambda_2 = 1.0$, assigning greater importance to carbon emissions than training time, reflecting the ecological focus of this study. Higher scores indicate a more favorable trade-off between predictive performance and resource consumption.

It should be noted that both $\hat{t}$ and $\hat{c}$ are inherently hardware- and location-dependent, as training time varies with the underlying hardware configuration and CO2 emissions are tied to the carbon intensity of the local electricity grid [1]. Consequently, EWF1 scores reflect the ecological efficiency of the specific experimental setup used in this study and may not generalize directly to other hardware or energy infrastructures.

4.5 Validation Strategy

To obtain reliable and generalizable performance estimates, a cross-validation strategy was employed rather than a single train-test split, as described in Section 2.x.

Specifically, K-Fold cross-validation was applied with $k = 5$ folds, a value widely recommended in the literature as a practical balance between computational cost and estimation stability [15]. Each model is trained and evaluated five times on non-overlapping subsets of the data, and the final performance score is averaged across all folds. This approach is consistent across all models and datasets, ensuring comparability of results.

To ensure reproducibility, a fixed random state was used for all cross-validation splits, model initialization, and data shuffling. The specific value and its technical implementation are described in Section 5.1.2.

4.6 Hyperparameter Tuning

To ensure a fair comparison across all models, hyperparameter tuning was applied to each model prior to evaluation. Without tuning, default parameters tend to favor certain model families over others, potentially skewing the comparison in terms of both predictive performance and resource consumption [16].

Hyperparameter optimization was performed using Optuna [17], an optimization framework based on Bayesian search (see Section 2.x).

Hier in 02 die genaue Erklärung

Grid search was considered but deemed infeasible given the scale of experiments: assuming five hyperparameters per model with ten candidate values each, evaluated across

three datasets and four models, a full grid search would require approximately 150,000 evaluations. Optuna was therefore configured to run 50 trials per model per dataset, striking a balance between tuning thoroughness and resource consumption, the latter being a relevant factor given that tuning runs contribute to the overall energy budget of this study, as described in Section 4.4.

Weighted F1-score was used as the optimization objective, as it accounts for class imbalance more robustly than accuracy, providing a more reliable estimate of model quality across datasets with varying class distributions.

Tuning was performed separately for each dataset, as the datasets differ substantially in size, feature space, and class distribution. A globally tuned model would not represent optimal performance on any individual dataset and would therefore introduce bias into the comparison.

Chapter 5

Implementation and Measurement

5.1 Experimental Setup

5.1.1 Hardware Setup

A critical decision in the experimental design was the selection of the computing platform on which all experiments would be conducted. Two options were considered: the university's shared computing infrastructure or a local machine. The latter was ultimately chosen, as it provides several key advantages for this study. First, it ensures complete reproducibility, since all experiments are executed on identical hardware configuration, eliminating interfering variables that could arise from shared resource conflicts or system variability. Second, it provides consistent availability and direct control over system configuration, which is particularly important given the extended runtime of the large-scale experiments (particularly on the 11M-instance HIGGS dataset). Third, local execution facilitates fine-grained performance monitoring and measurement of energy consumption, a primary objective of this study.

The hardware platform selected for all experiments is a consumer-grade desktop system with the following specifications: an NVIDIA RTX 4070 graphics processing unit (12 GB GDDR6 memory, 5,888 CUDA cores), an AMD Ryzen 7 5700X processor (8 cores / 16 threads, base clock 3.6 GHz), and 32 GB of DDR4 RAM. The system runs Windows 11 with all code implemented in Python 3.12.0.

Table 5.1: Hardware specification of the experimental system

Component	**Specification**
CPU	AMD Ryzen 7 5700X (8 cores / 16 threads, 3.6 GHz base)
GPU	NVIDIA RTX 4070 (12 GB GDDR6, 5,888 CUDA cores)
RAM	32 GB DDR4
Operating System	Windows 11

The choice of hardware reflects a deliberate trade-off between experimental scope and practical feasibility. The RTX 4070 represents a mid-to-high-end consumer GPU with

sufficient compute capability to execute GPU-accelerated training for XGBoost and MLP models in reasonable time, while remaining accessible to many practitioners. The Ryzen 7 5700X provides adequate CPU performance for baseline models (Logistic Regression, Random Forest) and CPU-based XGBoost training. Together, this configuration enables direct comparison of CPU- and GPU-accelerated training on the same platform, which is central to answering one of the research questions (*How does energy consumption differ between CPU-based and GPU-based training for equivalent tasks?*).

It is important to note that energy consumption and CO2 emissions measured in this study are specific to this hardware configuration and the local electricity grid, and may not directly generalize to other systems or geographic regions. However, the relative performance comparisons (e.g., the energy efficiency ranking of different algorithms) are more likely to generalize, as they reflect algorithmic properties rather than hardware specifics alone.

5.1.2 Software Environment

All experiments were implemented in Python 3.12.0. The main libraries used are described below.

Scikit-learn [18] served as the foundation of the evaluation pipeline. It provided the `LogisticRegression` and `RandomForestClassifier` implementations, as well as `KFold` and `cross_validate` for cross-validation, `StandardScaler` for feature normalization, `make_pipeline` for chaining preprocessing and model steps, and `f1_score` and `make_scorer` for evaluation metrics.

The XGBoost library [13] was used for the gradient boosting model, as it provides a highly optimized implementation that supports both CPU and GPU training.

For the MLP, PyTorch [19] was used as the deep learning backend. To integrate the PyTorch model into the scikit-learn evaluation pipeline, the Skorch library [20] was used, which wraps PyTorch models in a scikit-learn-compatible interface.

Hyperparameter tuning for all models except Logistic Regression was performed using Optuna, as described in Section 4.6.

Carbon emissions and energy consumption were tracked using CodeCarbon [21], which is described in detail in Section 5.6.

To ensure reproducibility, a fixed random seed of 42 was used for all data splits, cross-validation folds, and model initialization. A `requirements.txt` file is provided so that the exact library versions used in this study can be reproduced.

5.2 Dataset Integration

The three datasets used in this study — *Wine*, *Credit*, and *HIGGS* — were introduced in Section 4.2. This section describes how they are integrated into the software architecture, covering the central configuration file, the data loading pipeline, and dataset-specific constraints.

5.2.1 Configuration Architecture

To ensure a consistent foundation across all model scripts, a central configuration file (`config.py`) was created. It serves as a single source of truth for all dataset-related settings, so that each script accesses the same parameters without redundant definitions.

For each dataset, the configuration stores the file path, the name of the target column, and an optional row limit (`nrows`). The row limit was particularly relevant for the HIGGS dataset, which contains 11 million instances, allowing experiments to be run on smaller subsets where needed. Beyond these common fields, the configuration also captures dataset-specific properties. For example, the Credit dataset includes a header row that must be skipped during loading, and the Wine dataset does not contain column names in the source file, which are therefore defined in the configuration directly. Additionally, a label offset is applied to the Wine target column, as its class labels are encoded as 1, 2, and 3 rather than starting at 0, which is required by the classifiers used in this study.

The global constants `RANDOM_STATE` and `CV_FOLDS` are also defined here, ensuring that all scripts use identical values for random seeding and cross-validation, which is essential for the comparability of results.

5.2.2 Data Loading Pipeline

To read the datasets from their respective files, a custom `load_data` method was implemented. Depending on the dataset, the appropriate loading mechanism is selected. The *Wine* and *Credit* datasets are loaded using pandas' `read_csv` function. For the *Higgs* dataset, `read_parquet` is utilized. This was a deliberate decision to save storage space by storing the large *Higgs* dataset as a Parquet file rather than a standard CSV.

Following the initial loading phase, the *Higgs* data is reduced to a representative subset using a sampling approach controlled by the `nrows` parameter. Once the data size is managed, any columns specified in the configuration file are dropped from the respective dataset. The data is then split into the feature matrix X and the target vector y. If an offset is defined in the config, which is currently only the case for the *Credit* dataset, it is applied as described in the previous section. Finally, the method returns X and y for further use in the subsequent scripts. This process is visualized in Figure 5.1.

Due to the substantial scale of the *Higgs* dataset, several constraints were implemented to ensure the stability of the execution environment. Training a `Random Forest` model

on this specific data led to unmanageable processing times. For this reason, the `Random Forest` algorithm is explicitly bypassed for the *Higgs* dataset and the program terminates using `sys.exit(0)` to prevent system crashes. This allows the process to stay within the boundaries of the available hardware resources.

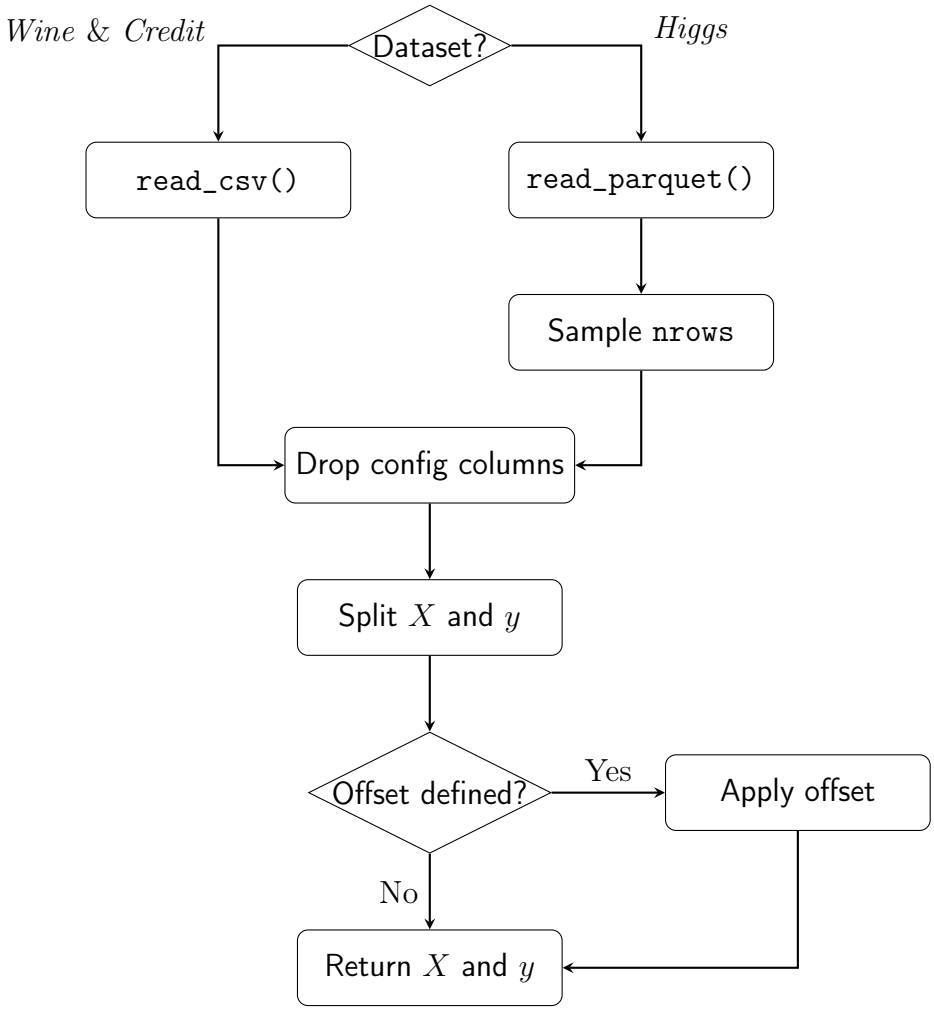

Figure 5.1: Visualization of the data loading pipeline and preprocessing steps within the `load_data()` method.

5.3 Reproducibility and Evaluation Design

To ensure fully reproducible results across all experiments, a global constant `RANDOM_STATE` with a value of 42 is utilized in every model script. This constant is applied to all functions requiring stochastic consistency. It is specifically passed to the `KFold` splitter, the `train_test_split` method, and the initialization of the machine learning models. For the multilayer perceptron, the random seed is explicitly set using `torch.manual_seed(RANDOM_STATE)` to guarantee deterministic weight initialization and training behavior.

The evaluation process relies on a standardized cross validation setup implemented uniformly across all scripts. The data is partitioned using `KFold` configured with five splits,

enabled shuffling, and the aforementioned random state. The execution is handled by the `cross_validate()` function.

The performance metrics are evaluated using a custom scoring dictionary. This dictionary tracks both the standard accuracy and a weighted F1 score constructed via `make_scorer(f1_score, average="weighted")`. Once the evaluation is complete, the overall performance is aggregated by calculating the mean of the individual fold results, which is accessed through `cv_results["test_accuracy"].mean()` alongside the corresponding F1 score array. Having defined the measures of reproducibility and testing, the focus now shifts to the actual implementation of the predictive models.

5.4 Model Implementations

As outlined in Section 4.3, the following classification algorithms were employed to address the research questions mentioned previously. This section therefore details the specific implementation of each model.

5.4.1 Logistic Regression

The first baseline model is a standard logistic regression. To ensure the model processes all features on an equal scale, the algorithm is embedded in a pipeline starting with a `StandardScaler`. This initial scaling step is crucial for the stability and overall performance of the subsequent classification.

The regression is configured to use the Stochastic Average Gradient (SAG) algorithm by setting the `solver` parameter to `sag`. This specific solver is highly optimized for large datasets. It significantly reduces the memory footprint and required training time, which is especially important when evaluating the extensive *Higgs* dataset. To ensure proper convergence, the maximum number of iterations is capped at 1000 using the `max_iter` parameter.

This model setup deliberately excludes any hyperparameter optimization. Instead, it acts as a stable baseline to evaluate the raw computational time without the extra overhead of complex search processes.

5.4.2 Random Forest

The second baseline algorithm is the Random Forest classifier. This method belongs to the family of ensemble learning techniques. Unlike logistic regression, this algorithm does not require prior feature scaling. To maximize computational efficiency, the execution is fully parallelized across all available CPU cores by setting the `n_jobs` parameter to `-1`. This configuration allows the processor to run at maximum capacity, which directly influences the subsequent energy consumption tracking and overall runtime measurements.

An important methodological decision is the explicit exclusion of the *Higgs* dataset for this specific classifier. As established in Section 5.2.2, training a Random Forest on such a massive volume of data leads to unmanageable runtimes. The primary research question focuses on the balance between energy consumption and predictive performance across varying dataset sizes and model complexities. Forcing this algorithm to process the *Higgs* data would result in a massive energy footprint and an excessive runtime. This extreme resource consumption would severely skew the comparative baseline without providing proportional insights into the underlying scaling behavior. Therefore, this execution is bypassed to maintain a sensible and fair evaluation framework.

To ensure maximum predictive capabilities for the remaining data, the model configurations were optimized using Optuna. The optimization process specifically targeted maximizing the weighted F1 score. For strict reproducibility, the final parameter combinations for the *Wine* and *Credit* datasets are documented in Table 5.2.

Table 5.2: Optimized hyperparameters for the Random Forest classifier obtained via Optuna optimization.

Parameter	*Wine*	*Credit*
`n_estimators`	221	381
`max_depth`	5	9
`min_samples_split`	8	10
`min_samples_leaf`	3	2
`max_features`	`sqrt`	`None`

hier noch die korrekten werte nach dem finalen run eintragen

5.4.3 XGBoost

The next tree-based ensemble model is the XGBoost classifier. This model serves as a direct point of comparison to the Random Forest. A primary reason for including XGBoost in this study is that it allows for a direct comparison of computational efficiency between CPU and GPU processing. To achieve this, the configuration dictionaries are kept identical for both executions. The only difference is the hardware device parameter, which is set to use the GPU. This controlled setup ensures a precise evaluation of the energy and emissions generated by the exact same model architecture on different hardware. This setup directly addresses the third research question: *How does energy consumption differ between CPU-based and GPU-based training for equivalent tasks?*.

Additionally, XGBoost has a specific technical requirement for handling target variables. While the Random Forest algorithm natively adapts to different class structures, XGBoost requires explicit instructions for its learning objective. As mentioned in Section 4.2, the *Wine* dataset consists of three distinct categories. Therefore, the objective parameter is set to `multi:softmax`, paired with the `mlogloss` evaluation metric and a specified

number of classes. In contrast, the *Credit* and *Higgs* datasets contain only two categories. Consequently, their objective is configured as `binary:logistic` and evaluated using the standard `logloss` metric.

Similar to the previous baseline algorithm, the parameters tailored to each dataset were determined using the Optuna optimization framework. Furthermore, the algorithm processes the unscaled feature matrix X directly. It also uses the globally defined random state parameter to guarantee a deterministic execution across all hardware setups. The resulting configurations used for the final training phase are detailed in Table 5.3.

Table 5.3: Optimized hyperparameters for the XGBoost classifier across all datasets.

Parameter	*Wine*	*Credit*	*Higgs*
`objective`	`multi:softmax`	`binary:logistic`	`binary:logistic`
`eval_metric`	`mlogloss`	`logloss`	`logloss`
`num_class`	3	N/A	N/A
`n_estimators`	464	108	475
`max_depth`	7	3	8
`learning_rate`	0.0139	0.2878	0.2428
`subsample`	0.7790	0.9640	0.7100
`colsample_bytree`	0.7500	0.8430	0.9368
`min_child_weight`	5	7	2

5.4.4 Multilayer Perceptron

As discussed in Section 4.3, a Multilayer Perceptron was selected to represent the neural network family. The implementation uses PyTorch as the primary deep-learning framework, combined with the Skorch library. Skorch wraps the PyTorch model in an interface that is fully compatible with the scikit-learn ecosystem. This is necessary because the entire evaluation logic, including cross-validation and pipeline construction, relies heavily on scikit-learn.

A dynamic architecture was chosen to ensure the network structure directly reflects the optimal parameters found during the Optuna tuning process. At runtime, the script reads the best parameter set from a JSON file generated during the tuning phase. It extracts the number of hidden layers via `n_layers` and the neuron count for each individual layer via `layer_i`. These values are then assembled into a list called `layer_sizes`. This list is subsequently passed to the `MLPModule` class. This class constructs the full network dynamically upon instantiation, making the architecture entirely data-driven.

The `MLPModule` inherits from `nn.Module` and builds the hidden layers by iterating over the previously mentioned list. Each hidden block consists of four sequential operations. It starts with a fully connected linear transformation using `nn.Linear`, followed by a batch normalization step using `nn.BatchNorm1d`, a ReLU activation function, and a final dropout layer. Batch normalization normalizes the activations across the current mini-batch. This

step stabilizes the training process and reduces internal covariate shifts [22]. The dropout layer randomly deactivates a fraction of neurons during the training phase. It serves as a regularization technique to prevent the network from memorizing the data [23]. After processing all hidden blocks, a single linear output layer maps the final representations to raw class logits. The dimension of this output equals the exact number of target classes. No activation function is applied to this final layer, because the `CrossEntropyLoss` function explicitly expects unnormalized inputs.

das alles ist ziemlich theoretisch und sollte in die 02 eher rein

To minimize the overall training time, the implementation checks for CUDA availability using `torch.cuda.is_available()`. The algorithm runs on the GPU if one is present and automatically falls back to the CPU otherwise. PyTorch requires highly specific numeric types, and neural networks react sensitively to implicit type mismatches. Therefore, the feature matrix X is explicitly cast to `float32` and the target vector y to `int64` before the training begins.

The neural network is embedded within a scikit-learn `Pipeline` together with a `StandardScaler`. Since Multilayer Perceptrons lack built-in scale invariance, standardizing the inputs to a mean of zero and a unit variance is necessary to keep the gradient flow stable. The `NeuralNetClassifier` provided by Skorch receives all architecture parameters through a specific naming convention. It uses the `module__*` prefix to forward keyword arguments directly to the constructor of the `MLPModule`. The Adam optimizer and the `CrossEntropyLoss` criterion are used consistently across all datasets. The learning rate and the batch size are taken directly from the tuning results and are therefore tailored to each dataset. A fixed random seed is applied using `torch.manual_seed` to guarantee exact reproducibility across all iterations.

The maximum number of training epochs is set to 200. This value acts as an upper boundary rather than a strict target. An early stopping callback (`skorch.callbacks.EarlyStopping`) monitors the validation loss after each individual epoch. It halts the training process immediately if no improvement is observed for 10 consecutive epochs. This mechanism prevents unnecessarily long training runs and provides an additional layer of protection against overfitting. Consequently, the upper limit of 200 epochs is rarely reached in practice.

hier vielleicht eine visualisierung der layers einfügen

5.5 Hyperparameter Optimization

5.5.1 Optuna as Tuning Framework

Nur Implementation, keine Begründung (die steht in Section 4.6): optuna.create_study(direction="maximize"). TPESampler(seed=RANDOM_STATE) für MLP (reproduzierbare Trial-Reihenfolge). study.optimize(objective, n_trials=50). Tuning-Ergebnisse werden direkt als Hyperparameter-Dicts in die jeweiligen Model-Scripts übernommen.

5.5.2 Search Spaces per Model

Suchräume für XGB (n_estimators, max_depth, learning_rate, subsample, colsample_bytree, min_child_weight), RF (n_estimators, max_depth, min_samples_split, min_samples_leaf, max_features), MLP (n_layers, layer_sizes, dropout_rate, lr, batch_size). Tabelle oder kompakte Auflistung.

5.5.3 Tuning Strategy for HIGGS

MLP-Tuning auf HIGGS: stratifiziertes Subset via –tune-sample-size statt vollem Dataset. Train-Test-Split (80/20) statt CV für Tuning (Begründung: Laufzeit). Ergebnisse in best_params_mlp_higgs.json.

5.6 Emissions Measurement

Energy consumption and CO_2 emissions were tracked using *CodeCarbon* as described in Section 2.2. For the hardware configuration used in this study, GPU power was measured directly via hardware counters, while RAM was estimated at 10W. CPU power could not be measured directly on Windows and was therefore estimated at 32.5W based on 50% of the AMD Ryzen 7 5700X TDP, as discussed in Section 2.2. Figure 5.2 shows a visual representation of the process, from pre-processing to saving the results. Prior to training, data is loaded from the respective dataset and, where required, features are normalized using `StandardScaler`. Specifically, scaling was applied to Logistic Regression and MLP, as these models are sensitive to the magnitude of input features. Tree-based models (Random Forest, XGBoost) do not require feature scaling and were trained on the raw input data.

Each training run was wrapped in an `EmissionsTracker` instance, which monitored energy consumption across the full 5-fold cross-validation procedure. Upon completion, the tracker was stopped and the resulting emissions value, alongside accuracy, weighted F1-score, training time, and dataset size, was saved to a central `results.csv` file via the

`save_results()` utility function. In addition, *CodeCarbon* writes its own emissions log to the `emissions/` directory, providing a detailed record of all tracked runs. The same tracking approach was applied during hyperparameter tuning, ensuring that the energy cost of the tuning process is also accounted for in the overall energy budget of this study.

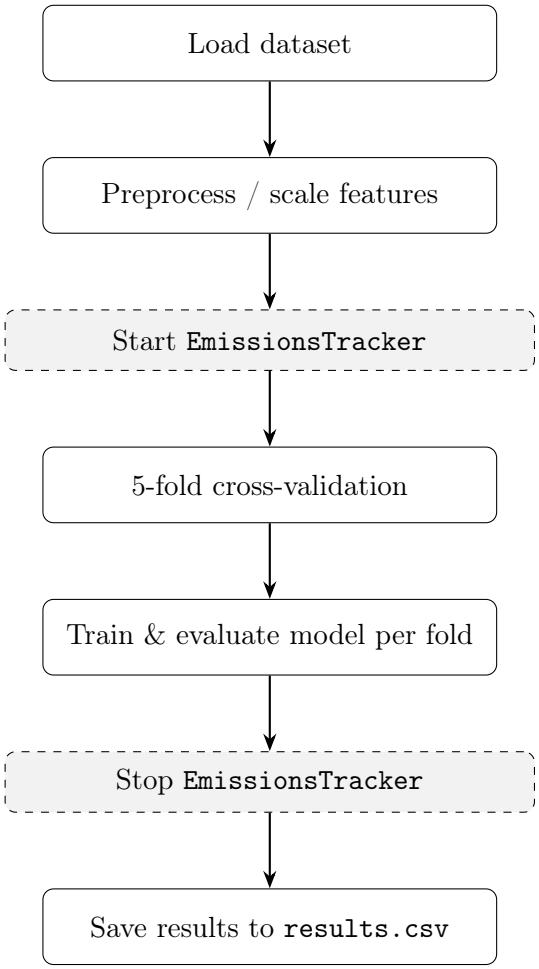

Figure 5.2: Training and emissions tracking procedure per model-dataset combination. Dashed boxes indicate CodeCarbon `EmissionsTracker` calls.

Ggf. noch Subsections ergänzen: 5.6.1 CodeCarbon Integration, 5.6.2 Measurement Granularity (pro Modell × Dataset, eindeutige project_names), 5.6.3 Limitations (CPU-Schätzung auf Windows, Background-Prozesse)

5.7 Inference Latency Measurement

Inference time was measured separately after training, outside the emissions tracker. For each model, a single sample from the test set was passed through the trained model and the elapsed time was recorded using `time.perf_counter()`. This provides a hardware-level estimate of per-sample prediction latency, which is relevant in deployment scenarios where repeated inference can accumulate significant energy costs [14].

Ergänzen: Begründung Single-Row statt Batch (Isolation von Training vs. Inference, kein Batch-Effekt). Modell kommt aus erstem CV-Fold (cv_results["estimator"][0]).

5.8 Results Persistence and Orchestration

5.8.1 Results Schema

results.csv Schema beschreiben: timestamp, model, dataset, nrows, accuracy, f1, co2eq_kg, training_time_s. Append-Logik (kein Überschreiben) — Hinweis auf mögliche Duplikate bei Mehrfachläufen. inference_time.csv analog. csv.DictWriter, Header-Check.

5.8.2 Experimental Orchestration

run_models.py: subprocess.run() pro Dataset × Modell-Kombination. Prozess-Isolation wichtig für CodeCarbon (prozessbasierte Messung). Fehlertoleranz: einzelne fehlgeschlagene Scripts blockieren nicht den Gesamtlauf.

Bibliography

[1] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Commun. ACM*, vol. 63, no. 12, pp. 54–63, Nov. 2020, ISSN: 0001-0782. DOI: 10.1145/3381831 Accessed: Mar. 19, 2026.

[2] E. Strubell, A. Ganesh, and A. McCallum, *Energy and Policy Considerations for Deep Learning in NLP*, Jun. 2019. DOI: 10.48550/arXiv.1906.02243 arXiv: 1906.02243 [cs]. Accessed: Mar. 19, 2026.

[3] A. Lacoste, A. Luccioni, V. Schmidt, and T. Dandres, *Quantifying the Carbon Emissions of Machine Learning*, Nov. 2019. DOI: 10.48550/arXiv.1910.09700 arXiv: 1910.09700 [cs]. Accessed: Mar. 19, 2026.

[4] *How Power Estimation Works in CodeCarbon - CodeCarbon*, https://docs.codecarbon.io/latest/expl estimation/. Accessed: Apr. 15, 2026.

[5] K. N. Khan, M. Hirki, T. Niemi, J. K. Nurminen, and Z. Ou, "RAPL in Action: Experiences in Using RAPL for Power Measurements," *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, vol. 3, no. 2, pp. 1–26, Jun. 2018, ISSN: 2376-3639, 2376-3647. DOI: 10.1145/3177754 Accessed: Apr. 15, 2026.

[6] C. Rodriguez, L. Degioanni, L. Kameni, R. Vidal, and G. Neglia, *Evaluating the Energy Consumption of Machine Learning: Systematic Literature Review and Experiments*, Aug. 2024. DOI: 10.48550/arXiv.2408.15128 arXiv: 2408.15128 [cs]. Accessed: Mar. 11, 2026.

[7] M. F. Stefan Aeberhard, *Wine*, 1992. DOI: 10.24432/C5PC7J Accessed: Mar. 19, 2026.

[8] I-Cheng Yeh, *Default of Credit Card Clients*, 2009. DOI: 10.24432/C55S3H Accessed: Mar. 19, 2026.

[9] P. Baldi, P. Sadowski, and D. Whiteson, "Searching for Exotic Particles in High-Energy Physics with Deep Learning," *Nature Communications*, vol. 5, no. 1, p. 4308, Jul. 2014, ISSN: 2041-1723. DOI: 10.1038/ncomms5308 arXiv: 1402.4735 [hep-ph]. Accessed: Mar. 19, 2026.

[10] D. Whiteson, *HIGGS*, 2014. DOI: 10.24432/C5V312 Accessed: Mar. 19, 2026.

[11] D. R. Cox, "The Regression Analysis of Binary Sequences," *Journal of the Royal Statistical Society. Series B (Methodological)*, vol. 20, no. 2, pp. 215–242, 1958, ISSN: 0035-9246. JSTOR: 2983890. Accessed: Apr. 8, 2026.

[12] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001, ISSN: 1573-0565. DOI: 10.1023/A:1010933404324 Accessed: Mar. 19, 2026.

[13] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16, New York, NY, USA: Association for Computing Machinery, Aug. 2016, pp. 785–794, ISBN: 978-1-4503-4232-2. DOI: 10.1145/2939672.2939785 Accessed: Mar. 19, 2026.

[14] R. Desislavov, F. Martínez-Plumed, and J. Hernández-Orallo, "Trends in AI inference energy consumption: Beyond the performance-vs-parameter laws of deep learning," *Sustainable Computing: Informatics and Systems*, vol. 38, p. 100 857, Apr. 2023, ISSN: 22105379. DOI: 10.1016/j.suscom.2023.100857 Accessed: Apr. 10, 2026.

[15] T. Hastie, R. Tibshirani, and J. Friedman, "Model Assessment and Selection," in *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, T. Hastie, R. Tibshirani, and J. Friedman, Eds., New York, NY: Springer, 2009, pp. 219–259, ISBN: 978-0-387-84858-7. DOI: 10.1007/978-0-387-84858-7_7 Accessed: Apr. 10, 2026.

[16] A. Bagnall and G. C. Cawley, *On the Use of Default Parameter Settings in the Empirical Evaluation of Classification Algorithms*, Mar. 2017. DOI: 10.48550/arXiv.1703.06777 arXiv: 1703.06777 [cs]. Accessed: Apr. 10, 2026.

[17] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD '19, New York, NY, USA: Association for Computing Machinery, Jul. 2019, pp. 2623–2631, ISBN: 978-1-4503-6201-6. DOI: 10.1145/3292500.3330701 Accessed: Apr. 1, 2026.

[18] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011, ISSN: 1533-7928. Accessed: Mar. 19, 2026.

[19] A. Paszke et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, Dec. 2019. DOI: 10.48550/arXiv.1912.01703 arXiv: 1912.01703 [cs]. Accessed: Mar. 19, 2026.

[20] B. Ghosh, *Skorch: The Bridge Between PyTorch and Scikit-Learn*, Sep. 2024. Accessed: Mar. 26, 2026.

[21] B. Courty et al., *Mlco2/codecarbon: V2.4.1*, Zenodo, May 2024. DOI: 10.5281/zenodo.11171501 Accessed: Apr. 15, 2026.

[22] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proceedings of the 32nd International Conference on Machine Learning*, PMLR, Jun. 2015, pp. 448–456. Accessed: Apr. 18, 2026.

[23] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014, ISSN: 1533-7928. Accessed: Apr. 18, 2026.

List of Figures

List of Tables